

# UD3 · Ejercicios

Programación embebida en páginas web

**Entrega.** Guarda cada ejercicio en su propia carpeta dentro de ud3, en el repositorio de ejercicios. Incluye las respuestas a las preguntas en el README del proyecto de cada ejercicio, de acuerdo con las normas de entrega.

## 3.1 · Controlador y vistas

Parte del ejercicio 2.7 (la web estática con las páginas de inicio, destacados, galería) y conviértelo en una aplicación en la que las páginas se sirvan desde un controlador.

### Qué debes hacer

- Copia el proyecto del ejercicio 2.7 en una carpeta nueva.
- Modifica el código y crea los elementos necesarios para que las páginas sean servidas por un controlador.
- Actualiza el menú para que sus enlaces apunten a las rutas elegidas
- Extrae el menú repetido a un fragmento de Thymeleaf e inclúyelo en las tres vistas.

### Contesta en el README

- ¿Por qué has tenido que cambiar de ubicación las vistas?
- ¿Has tenido que cambiar el código HTML del menú de navegación? ¿Por qué?
- ¿Tienen que llamarse igual la ruta de un @GetMapping y la vista que devuelve? Justifícalo con tu proyecto.

## 3.2 · Listado de números aleatorios

Crea una aplicación que atienda GET /list y muestre un listado de números aleatorios

### Funcionamiento

- El controlador genera una lista con una cantidad aleatoria de elementos, entre 0 y 50, y cada elemento es un número aleatorio entre 1 y 100.
- Se muestran los números en una tabla, con una fila por número, y debajo el texto «Total números: N».
- Si la lista está vacía, no se muestran la tabla ni el total, y aparece el mensaje «No se ha generado ningún número»
- Los números menores de 50 se mostrarán con color de letra verde oscuro sobre fondo verde claro.
- Los números mayores o iguales a 50 con color de letra rojo y fondo rosa
- Deberás crear las clases CSS que contengan los atributos de los dos puntos anteriores. Elige la clase según el valor de cada número. No uses estilos escritos en línea ni calcules el color en Java.



### 3.3 · Catálogo de módulos

---

Crea una aplicación que muestre los módulos de un ciclo formativo con un listado y un filtro.

- Datos: una lista en memoria, en el controlador, de objetos Modulo con id, nombre, curso (1 o 2) y horas. Introduce mínimo 6 módulos, debe haber de los dos cursos.
- GET /modulos: listado con el nombre, el curso y las horas de cada módulo. Si llega el parámetro opcional curso filtrará el listado y solo muestra los de ese curso. Por ejemplo /modulos?curso=1.
- Encima del listado, tres enlaces: «Todos», «1.º» y «2.º», contruidos con @{/modulos(curso=...)} o @{/modulos}.

#### Contesta en el README

- ¿Cómo has declarado el parámetro curso en el método? ¿Qué ocurriría si no lo hubieras hecho opcional y visitaras /modulos?

### 3.4 · Detalle del módulo

---

Copia el proyecto anterior en una carpeta nueva y añade la página de detalle de cada módulo.

En el listado, convierte el nombre de cada módulo en un enlace a su detalle GET /modulos/{id}

- Muestra la página de detalle con los datos del módulo e incluye un enlace para volver al listado.
- Si el id no existe, redirige a /modulo-no-encontrado. Esa ruta muestra una página propia con un mensaje y un enlace para volver al listado

### 3.5 · Formulario de inscripción

---

Crea una aplicación para inscribirse en una actividad del centro (una excursión, un taller...).

Peticiones que debe atender el controlador:

- GET /inscripciones listado de inscripciones
- GET /inscripciones/nueva muestra el formulario vacío
- POST /inscripciones crea una inscripción, le asigna un id y la guarda en una lista en memoria. Después redirige a GET /inscripciones, que muestra el listado.
- Añade un menú con enlace al formulario para crear una nueva inscripción y otro al listado de inscripciones.

El listado de inscripciones debe mostrar una tabla con las inscripciones realizadas, mostrando el nombre y correo electrónico.

Las inscripciones se almacenan en memoria y deben tener los siguientes campos: nombre (texto), correo (tipo email), turno (mañana o tarde), comentarios (área de texto) y aceptaNormas (boolean).

### 3.6 · Edición y eliminación de inscripciones

---

Copia el proyecto anterior en una carpeta nueva y continúa sobre esa copia.

Reutiliza la plantilla del formulario para modificar una inscripción existente y añade la opción de eliminarla.

En el listado, cada inscripción debe contar con:

- Un enlace Editar a GET /inscripciones/{id}/editar. El controlador busca la inscripción y muestra el formulario con sus datos.
- Un botón Eliminar en un formulario que envíe un POST /inscripciones/{id}/eliminar, el controlador borrará en memoria la inscripción indicada

Además es necesaria una nueva ruta en el controlador. POST /inscripciones/{id}/editar, que actualiza la inscripción indicada y redirige al listado.

Se debe emplear el mismo formulario que en el proceso de alta, el título y la acción cambian según se trate de un alta o una edición. Si el id no existe, muestra una página propia de inscripción no encontrada.

### 3.7 · Validación del alta y la edición

---

Copia el proyecto anterior en una carpeta nueva y continúa sobre esa copia.

Añade la dependencia de validación y aplica las siguientes reglas:

| Campo        | Regla                               |
|--------------|-------------------------------------|
| nombre       | Obligatorio, máximo 60 caracteres   |
| correo       | Obligatorio y con formato de correo |
| turno        | Debe seleccionarse una opción       |
| comentarios  | Opcional, máximo 300 caracteres     |
| aceptaNormas | Debe estar marcada                  |

- Si hay errores, no guardes los cambios y vuelve al mismo formulario con los valores introducidos. Muestra un mensaje junto a cada campo incorrecto (th:errors) y resalta el control (th:errorclass).
- Si los datos son correctos, guarda el alta o la edición y redirige al listado.

#### Contesta en el README

- Con las herramientas de desarrollo del navegador, cambia type="email" por type="text" y envía un correo inválido. ¿Sigue actuando la validación del servidor? ¿Qué demuestra la prueba?